U.PORTO FEUP FACULDADE DE ENGENHARIA UNIVERSIDADE DO PORTO

# Developing a local-first shopping list application

FEUP-M.EIC004 – Large-Scale Distributed Systems; Group 6 - Winter 2025

Final Delivery

*Developing a Dynamo-like system for persistent availability in an online-first world.*


***João Torres*** *- up202205576*

***Ricardo Ramos*** *– up202206349*

***Ricardo Peralta*** *– up202206392*

# Project Overview & Goal

### The Challenge (The Problem)

- Design a high-availability, collaborative shopping list application.
- The system must be engineered to handle **millions of concurrent users** without data access bottlenecks.
- The primary challenge is managing **concurrent changes** (e.g., adding/deleting items) to the same list by multiple users

### The Architecture (The Solution: Local-First)

- The application is fundamentally **local-first.**
- Code runs on the user's device and **persists data locally.**
- This ensures the application is **fast**, **highly responsive**, and continues to work even when offline.

### The System (The Cloud)

A cloud component is implemented to provide two key functions:

- **Data Sharing:** Enables collaboration by sharing the list data among users who know the list's unique ID.
- **Backup Storage:** Provides a persistent, distributed backup of all shopping lists.

# Functional Requirements

*The application's fundamental requirement is to deliver a **fast, local-first experience** while supporting **highly-available, concurrent collaboration** among multiple users on a single shopping list, complete with shared item state and quantity tracking.*

## List Management & Sharing:

- Users must be able to create a new shopping list via the user interface.
- Upon creation, the list gets a **unique ID** (e.g., the name) that can be shared.
- Any user with the list's unique ID is always permitted to access and modify the list.

## Concurrency:

- The system must support **concurrent changes** to the list (Add and Delete items) by multiple users at the same time.
- The primary goal of the design is to ensure **high availability** despite concurrent changes.

## Item State & Attributes:

- Each item must support basic addition and deletion.
- Each item can be associated with a **Check-off Flag**, which is marked after the item is acquired.
- Each item can be tracked with a **Quantity Counter**, which includes a target quantity and how many items are already acquired.

# Design Challenge: High Scale & Availability

**The Problem:**

Support millions of concurrent users while avoiding data access bottlenecks and eliminating single points of failure.

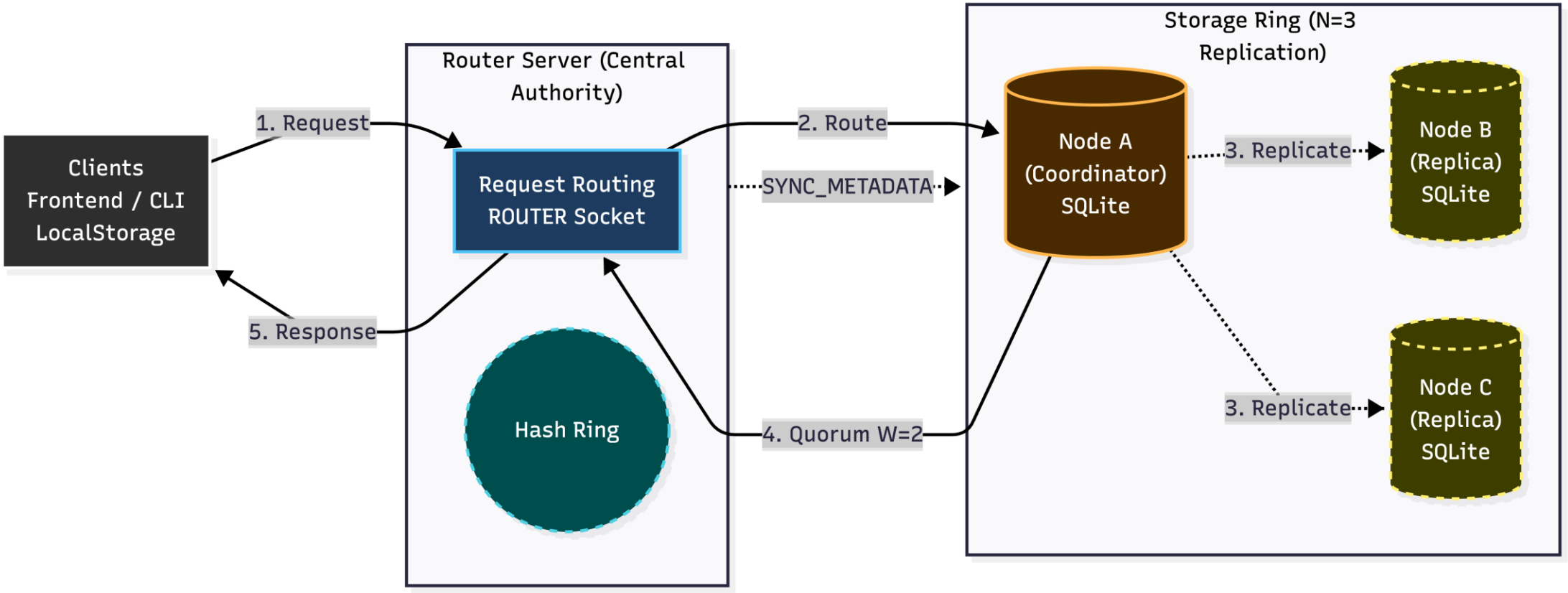
| Challenges | Solution |
|---|---|
| Single point of failure | Multiple Node Servers with data replication |
| Coordinator overload | Router distributes requests using consistent hashing<br>Nodes can be dynamically added to the hash ring. |
| Thread blocking | ZeroMQ async messaging with DEALER/ROUTER pattern in Router and REQ/REP in nodes with thread management to be non-blocking. |
| Node failures | Heartbeat monitoring + automatic node removal |

**Key Design Decisions:**

- **Replication Factor (N=3):** Each list is replicated to 3 nodes
- **Quorum-based operations:** R=2 for reads, W=2 for writes
- **Automatic failover:** Dead nodes detected via heartbeat timeout (15s)
- Router acts as a central authority of the nodes' positions, but the nodes are self-coordinating. While the Router is a single point of failure, nodes and web client can continue operating if the router is down and **administrator** handles the restarts on failure.

# Cloud Architecture: Dynamo-Style

- Architecture Overview:



# Design Choice: Data Sharding

**Consistent Hashing with Virtual Nodes**

**Why Consistent Hashing?**

- Adding/removing nodes only affects neighbors
- Predictable load distribution
- Keys naturally distribute across the ring

**Virtual Nodes:**

- Each physical node owns **4 virtual nodes** on the ring
- VNode ID format: (e.g. abc123:0, abc123:1, ... )

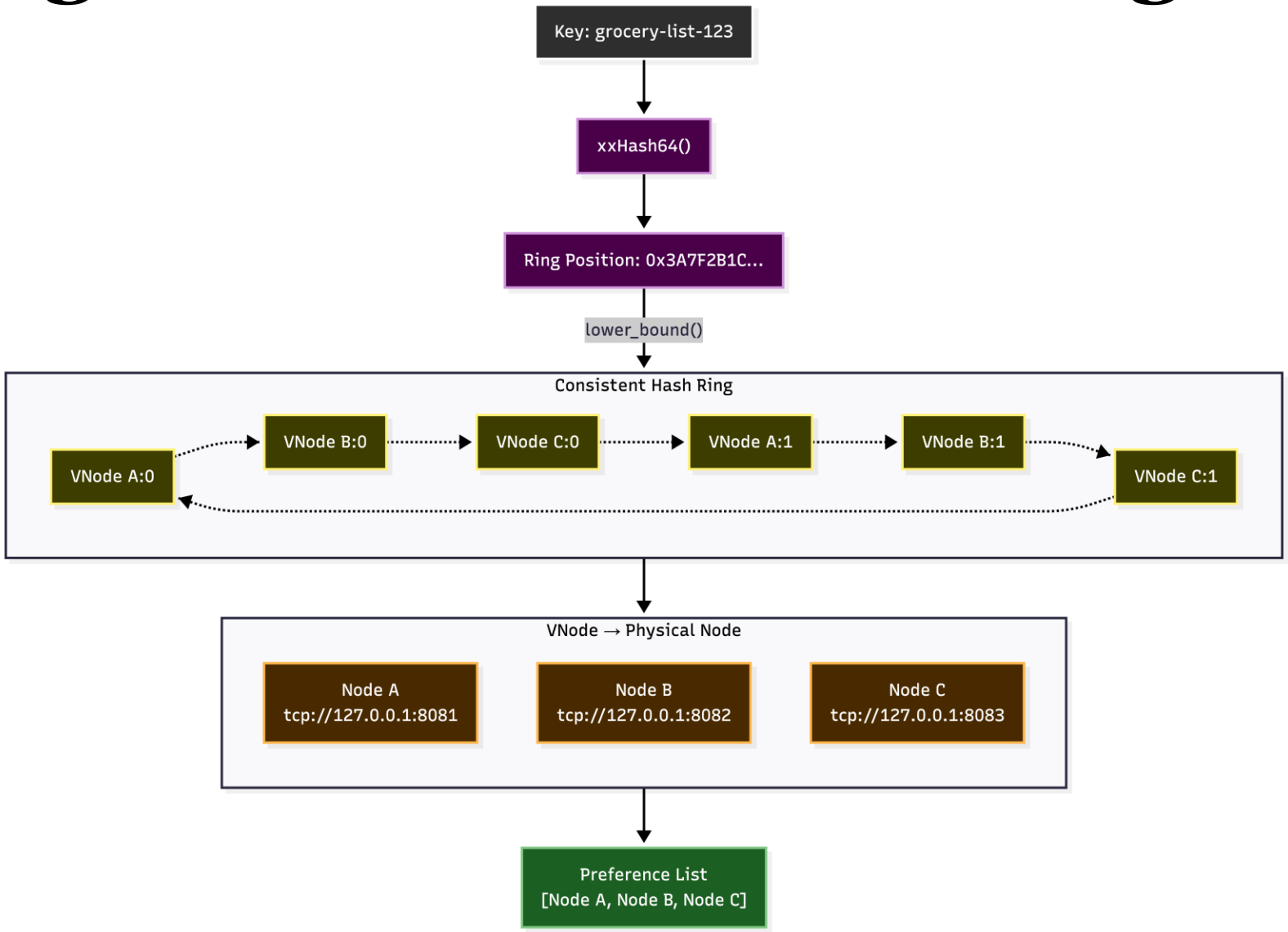
# Design Choice: Administrator

An administrator process which can interact with our system helps manage the state of our system.

**Key features:**

- Awareness of the hash ring organization.
- Manage health of processes (router and nodes).
- Inspection of individual nodes.
- Automatic restart of processes.
- Manual shutdown and launch of nodes.

# Design Choice: Data Sharding

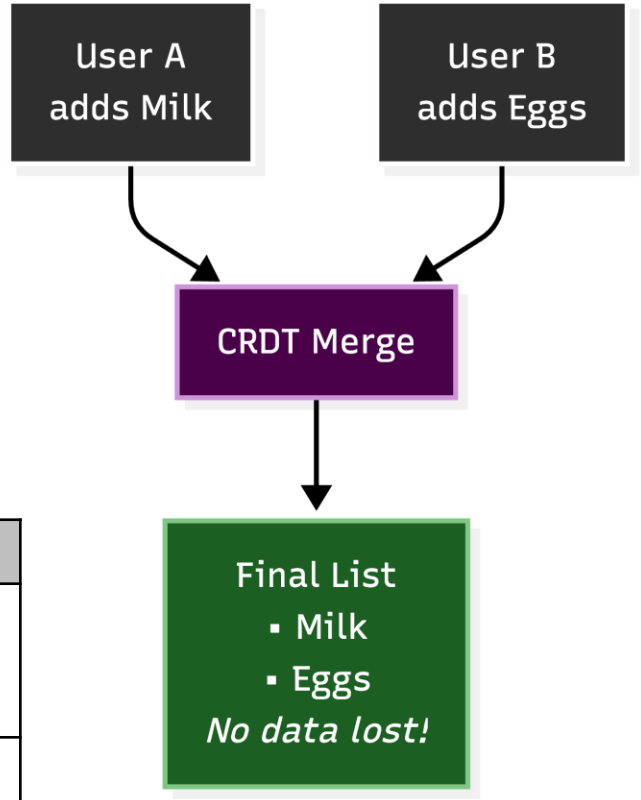


# Design Challenge: Consistency Management

### The Consistency Problem

Multiple users editing the same list simultaneously:

- User A adds "Milk" while offline
- User B adds "Eggs" at the same time
- Both come online and sync → **No data loss occurs!** CRDTs are built to be idempotent, associative and commutative.

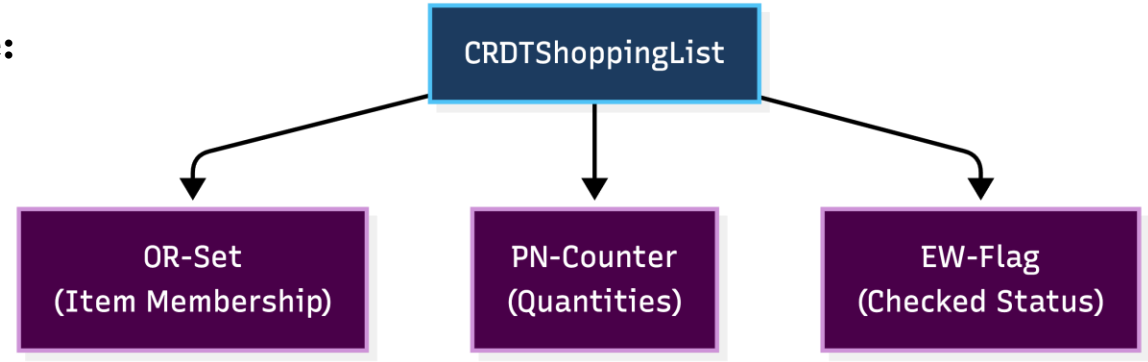



### Solution: Custom CRDTs (Conflict-free Replicated DataTypes)

| CRDT Type | Purpose | Implementation |
|---|---|---|
| **OR-Set** | Item membership (add/delete) | Tag-based: tracks **adds** and **removes** per item |
| **PN-Counter** | Quantities | Separate maps for **increments** and **decrements** per node |
| **Enable-Wins-Flag** | Checked status | Disable invalidates any enable. However, any enable that was not there will result in flag being enabled. |

# Deep Dive: CRDT Data Model

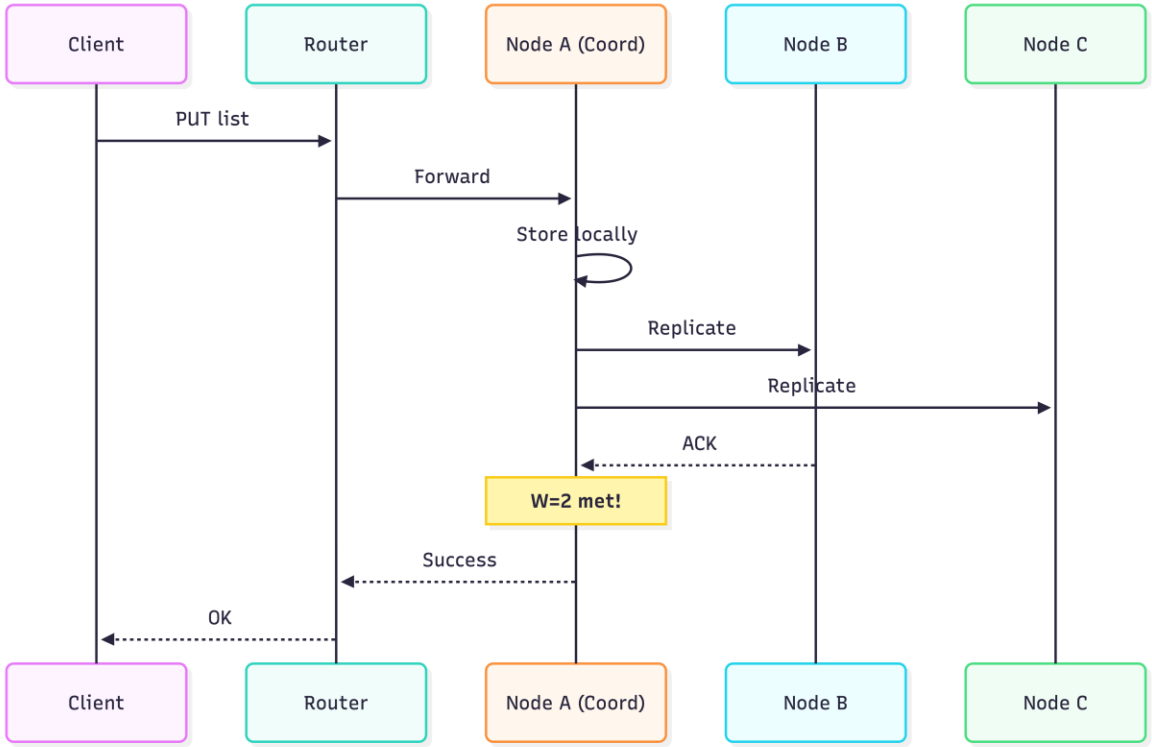
- **Shopping List Structure:**




- **CRDT Types Explained:**

| CRDT Type | State | Operations | Merge Rule |
|---|---|---|---|
| **OR-Set** | *adds{item→tags}, removes{item→tags}* | *add(item, tag), remove(item)* | Union of maps; visible if ***adds \ removes ≠ ∅*** |
| **PN-Counter** | *inc{node→val}, dec{node→val}* | *increment(), decrement()* | Max per node, then ***Σinc - Σdec*** |
| **EW-Flag** | *enables{tags}, disables{tags}* | *enable(tag), disable()* | Union sets; enabled if any ***enables \ disables ≠ ∅*** |

# Replication




**Read Repair**

- Coordinator queries all N replicas in parallel
- Returns after R=2 responses (quorum)
- If versions differ → **merge & read-repair** missing/stale replicas
- Client gets latest merged state even if quorum is not achieved.

**Topology change:**

- New nodes request from neighbors for any replicas they contain and should be owned by them.
- Nodes push replicas they were responsible for to their new owners.

# Implementation & Tooling

**Tech Stack**

| Component | Technology |
|---|---|
| **Backend** | C++20 |
| **Messaging** | ZeroMQ (DEALER/ROUTER, REQ/REP) |
| **Hashing** | xxHash |
| **Storage** | SQLite (per-node) |
| **Serialization** | nlohmann/json |
| **Frontend** | Next.js + CLI |

**Testing**

- Unit tests for CRDTs (Google Test)
- System integration tests
- Conflict scenario validation

**Architecture Components**

| Component | Purpose |
|---|---|
| `RouterServer` | Request routing, membership management |
| `NodeServer` | Data storage, quorum coordination |
| `AdminConsole` | Process management, monitoring |
| `ClientCLI` | Terminal-based CRUD operations |
| `CRDTShoppingList` | Custom CRDT implementation |

# Conclusion

- Goals achieved

A dynamo-inspired distributed architecture was developed. The application works completely offline using local storage and synchronizes that information when a backend is available.
Conflict resolution is ensured out-of-the-box by structuring our lists according to CRDT. The high availability requirement was met by data replication across nodes, sloppy quorum and data distribution using consistent hashing.
While making the system distributed and highly available implied it was not desired to have strong consistency of data (larger communication overhead), our system still ensures eventual consistency of data.
Future work to develop at this project would involve removing the router as a single point of failure in communication, optimizing the CRDTs with tombstone management/garbage collection, load testing the system or load balancing of the virtual nodes.